FLARE: FLOW-LINKED ALERT REVIEW FOR LEAKAGE-SAFE, CASE-LEVEL DETECTION OF ILLICIT BITCOIN FLOWS

Sourav Choudhary

Roll No. 23HS23017

FLARE: FLOW-LINKED ALERT REVIEW FOR LEAKAGE-SAFE, CASE-LEVEL DETECTION OF ILLICIT BITCOIN FLOWS

Report submitted to
Indian Institute of Technology, Kharagpur
for the award of the degree

of

Master of Science in Economics

Bachelor Thesis Project (BTP-1)

by

Sourav Choudhary (23HS23017)

Under the guidance of

Prof. Gourishankar S. Hiremath



DEPARTMENT OF HUMANITIES AND SOCIAL SCIENCES
INDIAN INSTITUTE OF TECHNOLOGY, KHARAGPUR

November 2026

© 2026 Sourav Choudhary. All rights reserved.

CERTIFICATE

This is to certify that the project report entitled **"FLARE: Flow-Linked Alert Review for Leakage-Safe, Case-Level Detection of Illicit Bitcoin Flows"**, submitted by **Sourav Choudhary** (Roll No. 23HS23017) to the Indian Institute of Technology, Kharagpur, India, is a record of bona fide project work carried out by him under my supervision and guidance during the Autumn Semester 2026-27, and is worthy of consideration for the Bachelor Thesis Project (BTP-1) towards the degree of Master of Science in Economics of the Institute.

_____

Prof. Gourishankar S. Hiremath
Department of Humanities and Social Sciences
Indian Institute of Technology, Kharagpur

Date: _____

Place: Kharagpur

DECLARATION

I certify that

(a) the work contained in this report is original and has been done by me under the guidance of my supervisor.

(b) the work has not been submitted to any other Institute for any degree or diploma.

(c) I have followed the guidelines provided by the Institute in preparing the report.

(d) I have conformed to the norms and guidelines given in the Ethical Code of Conduct of the Institute.

(e) whenever I have used materials (data, theoretical analysis, figures, and text) from other sources, I have given due credit to them by citing them in the text of the report and giving their details in the references. Further, I have taken permission from the copyright owners of the sources, whenever necessary.

_____

Signature of the Student
Sourav Choudhary (23HS23017)

Date: _____

Place: Kharagpur

ACKNOWLEDGEMENT

I am grateful to my supervisor, **Prof. Gourishankar S. Hiremath**, Department of Humanities and Social Sciences, IIT Kharagpur, for accepting this project and for his guidance on framing a research question that connects machine learning with the economics of financial regulation and compliance. His emphasis on careful empirical design shaped the evaluation protocol of this work.

I thank the Department of Humanities and Social Sciences for the academic environment and the facilities provided. I also acknowledge the authors of the Elliptic and Elliptic++ datasets for releasing them for research, the open-source communities behind Python, PyTorch, LightGBM, scikit-learn and NetworkX, and Kaggle for the free GPU notebooks on which all experiments were run.

Finally, I thank my family and friends for their constant encouragement and support.

Sourav Choudhary
23HS23017
IIT Kharagpur, November 2026

ABSTRACT

Anti-money-laundering (AML) teams cannot investigate every alert, so a detection model is only as useful as the way it fills a limited review budget. Models that score transactions one at a time hand analysts lists of isolated alerts, even when several belong to the same laundering flow. This report proposes FLARE (Flow-Linked Alert Review), which changes the unit of review from transactions to cases. On the public Elliptic++ Bitcoin dataset (203,769 transactions, 822,942 addresses, 49 time steps), high-risk transactions are expanded along money-flow and shared-address links, with hop, hub and size limits, into compact flow groups that are reviewed whole under the same fixed budget. All experiments follow a strict chronological protocol with point-in-time features. An audit shows that the official Elliptic++ wallet features are lifetime totals that leak future information; leak-free replacements perform as well or better. LightGBM is the strongest transaction scorer (test PR-AUC 0.750), ahead of a direction-aware GraphSAGE (0.720). At a 1% budget, FLARE queues find as many illicit transactions as the transaction queue (210 versus 208) with 56% fewer separate cases (36 versus 84 per 100 reviews; paired-bootstrap 95% CI of the difference [-400, -212] cases). With 10-20% of the training labels, FLARE finds 15-17% more illicit transactions, and with GraphSAGE scores it finds significantly more at most budgets. Gradient-selected evidence edges lower group scores two to four times more than random edges when deleted. The extra score terms do not beat a plain sum of member risk, and connected components are an equally strong grouping baseline. Code and results are public at https://github.com/Onethybeing/flare.

Keywords: anti-money laundering; Bitcoin; Elliptic++; graph neural networks; LightGBM; review queues; data leakage; explainability; label scarcity; financial regulation.

CONTENTS

Certificate . . . . . **ii**

Declaration . . . . . **iii**

Acknowledgement . . . . . **iv**

Abstract . . . . . **v**

List of Figures . . . . . **viii**

List of Tables . . . . . **ix**

List of Abbreviations and Symbols . . . . . **x**

Chapter 1 Introduction **1**

1.1 Background and motivation 1

1.2 Problem statement and the FLARE approach 1

1.3 Research questions 2

1.4 Contributions 2

1.5 Scope 3

1.6 Organisation of the report 3

Chapter 2 Literature Review **4**

2.1 Machine learning for AML on public blockchains 4

2.2 Graph learning for fraud and anomaly detection 4

2.3 Recent temporal and multi-view models on Elliptic 4

2.4 Granularity of review queues 5

2.5 Explainability 5

2.6 Evaluation methodology 5

2.7 Research gap 6

Chapter 3 Data and Leakage Audit **7**

3.1 The Elliptic++ dataset 7

3.2 Temporal structure and labels 7

3.3 Finding 1: all money-flow edges lie within a single time step 8

3.4 Finding 2: the official wallet features are lifetime summaries 8

3.5 Implications for the design 9

Chapter 4 Methodology **10**

4.1 Overview . . . 10
4.2 Problem formulation . . . 11
4.3 Point-in-time address features . . . 11
4.4 Risk models . . . 11
4.4.1 Tabular models . . . 11
4.4.2 Direction-aware GraphSAGE . . . 12
4.4.3 Wallet model . . . 12
4.5 Flow-constrained candidate groups . . . 13
4.6 Group scoring . . . 13
4.7 Review queues and evaluation metrics . . . 14
4.8 Counterfactual evidence test . . . 15
4.9 Experimental protocol . . . 16

Chapter 5 Results and Discussion . . . 17
5.1 Transaction-level detection (RQ1) . . . 17
5.2 Cross-view features and leakage (RQ2) . . . 19
5.3 Case-level review queues (RQ3) . . . 20
5.4 Group-level proxy quality . . . 21
5.5 Robustness: label scarcity and tuning (RQ4) . . . 23
5.6 Counterfactual evidence (RQ5) . . . 24
5.7 Case study . . . 25
5.8 Discussion . . . 26
5.8.1 Economic interpretation . . . 26
5.8.2 Why grouping helps . . . 27
5.8.3 Negative results and threats to validity . . . 27

Chapter 6 Conclusions and Future Scope . . . 28
6.1 Summary of findings . . . 28
6.2 Limitations . . . 28
6.3 Future scope (plan for BTP-2) . . . 29
6.4 Concluding remarks . . . 29

References . . . 30

Appendix A: Hyperparameters and Tuned Configurations . . . 32

Appendix B: Additional Results . . . 33

Appendix C: Software and Reproducibility . . . 34

LIST OF FIGURES

Figure 3.1 Transactions per time step by label 8

Figure 3.2 Illicit share of labelled transactions per time step 8

Figure 4.1 System pipeline 10

Figure 4.2 Chronological split and the difference between as-of features (current and earlier steps... 12

Figure 4.3 Group construction on a toy step: (a) risk scores, (b) seeds, (c) hop-limited expansion... . 14

Figure 4.4 Transaction queue versus group queue for the same budget of ten reviews 15

Figure 5.1 Test PR-AUC of all model and feature-view combinations (mean ± standard deviation over seeds) 18

Figure 5.2 Illicit F1 per test time step (seed 0) 18

Figure 5.3 (a) LightGBM PR-AUC by transaction feature view 19

Figure 5.4 Review queues versus budget with LightGBM risk: illicit transactions found, cases per 100... 21

Figure 5.5 Review queues versus budget with GraphSAGE risk 22

Figure 5.6 FLARE minus transaction queue: total difference over the 14 test steps with 95%... . . . 22

Figure 5.7 Illicit share of labelled members in top groups versus size-matched random groups; labels...

Figure 5.8 Label scarcity: transaction PR-AUC (left), illicit found (centre) and cases per 100 reviews...

Figure 5.9 Drop in group score after deleting m evidence edges selected by gradient, by the... . . . 26

Figure 5.10 A flow group from test step 37: twelve transactions, three known illicit (red) and nine... 26

Figure 6.1 Wireframe of a case-review console: ranked cases, the case graph with counterfactual... . 29

LIST OF TABLES

Table 2.1 Positioning of this work against the closest related studies 6
Table 3.1 Elliptic++ files and counts as loaded 7
Table 3.2 Leakage audit of the official Elliptic++ wallet features 9
Table 4.1 Experiments and the research questions they address 16
Table 5.1 Transaction-level results on test steps 36-49 (mean over seeds, with standard deviation for... 17
Table 5.2 Wallet-level results on test steps (three seeds) 19
Table 5.3 Review queues on test steps 36-49 at a 1% budget per step 20
Table 5.4 Ablations of the FLARE queue at a 1% budget 23
Table 5.5 Proxy quality of the top-10 groups per test step (140 groups per method) 23
Table 5.6 Label scarcity (LightGBM risk; mean ± standard deviation over three label subsets) . . . 24
Table 5.7 Counterfactual deletion test on the top GraphSAGE flow groups (score drop after deleting m... 25
Table A.1 Hyperparameters 32
Table A.2 As-of address features (each aggregated by mean and max over input and output addresses) 33
Table A.3 Group configurations selected on validation steps 31-35 (main run) 33
Table B.1 Illicit transactions found by each queue and budget (test steps 36-49) 33
Table B.2 Remaining transaction-level model results 34
Table C.1 Repository structure 34

LIST OF ABBREVIATIONS AND SYMBOLS

Abbreviation / symbol	Meaning
AML	Anti-money laundering
AF	All 165 original Elliptic features (local + aggregate)
AUG	17 augmented transaction features added by Elliptic++
AUC	Area under the ROC curve
CI	Confidence interval
F1	Harmonic mean of precision and recall
GNN	Graph neural network
LF	93 local transaction features
LR	Logistic regression
MLP	Multi-layer perceptron
P@K	Precision among the K highest-scored test transactions
PR-AUC	Area under the precision-recall curve (average precision)
RF	Random forest
XV	As-of cross-view (address) features aggregated to transactions
XV-lifetime	The same aggregation of the official (lifetime) wallet features
Symbols	
t, V_t	time step and its set of transactions
E_t	directed money-flow edges in step t
y_i	label: 1 illicit, 0 licit, -1 unknown
r_i	risk score of transaction i (model output in [0, 1])
b, B_t	budget fraction and review capacity of step t
G	a candidate group (set of transactions)
$S(G)$	group score
σ, q, h	seed fraction, expansion quantile, hop limit
H, M	hub degree limit (30), group size cap (40)
β, γ, δ	weights of motif, exposure and size terms in S(G)
$c(R)$	number of connected components (cases) in reviewed set R
w_e	weight of edge e in the GraphSAGE aggregation

Chapter 1
Introduction

1.1 Background and motivation

Money laundering disguises the origin of criminal proceeds by moving them through chains of transfers until they appear legitimate. Public cryptocurrencies such as Bitcoin [1] make this movement observable: every transaction and the addresses it touches are recorded on a public ledger. This visibility has made Bitcoin a standard test bed for machine-learning approaches to anti-money laundering (AML), starting with the Elliptic dataset of Weber *et al.* [2] and its extension Elliptic++ by Elmougy and Liu [3], [4].

From an economic point of view, AML is a problem of allocating a scarce resource. Compliance units and investigators have a fixed capacity; every alert they open costs analyst time, and every illicit flow they miss carries regulatory, reputational and social costs. The useful question for a detection system is therefore not only *how accurately it classifies*, but *how much illicit activity it surfaces per unit of investigation effort*. Most published models, however, score individual transactions and are evaluated with global metrics such as F1 or ROC-AUC [2], [5], [6]. When their top-ranked transactions are handed to an analyst, related transactions from the same laundering flow arrive as separate, disconnected alerts, and the analyst has to rediscover the links between them.

A second concern is evaluation validity. Transaction graphs evolve over time, and features that summarise an entity's whole history can silently include information from the future. Such leakage [7] inflates offline results and gives a misleading picture of how a model would perform when deployed prospectively.

1.2 Problem statement and the FLARE approach

This report studies how to turn model scores into *investigation cases* under a fixed review budget, using only information that would have been available at the time of review. Concretely, at each time step the analyst can review a fixed share of that step's transactions (0.5% to 5%). A *transaction queue* reviews the highest-risk transactions. A *group queue* first assembles high-risk transactions into connected groups along money-flow and shared-address links, and then reviews whole groups in score order. This pipeline is called **FLARE** (Flow-Linked Alert Review): alerts are linked along money flows and reviewed together as cases. The two queues are compared on how many known-illicit transactions

they find and on how fragmented the reviewed set is, measured as the number of separate cases (connected components) per 100 reviews.

1.3 Research questions

1. **RQ1 (detection).** How well do tabular and graph models detect illicit transactions under a strict chronological protocol, and does point-in-time address (cross-view) information help?
2. **RQ2 (leakage).** Do the official Elliptic++ wallet features contain future information, and how does replacing them with point-in-time features change wallet-level detection?
3. **RQ3 (cases).** At a fixed review budget, do group queues find as many illicit transactions as transaction or wallet queues, and how much less fragmented are they?
4. **RQ4 (robustness).** Does the result hold with fewer training labels and under different tuning choices?
5. **RQ5 (evidence).** Do gradient-selected evidence edges pass a counterfactual deletion test?

1.4 Contributions

The contributions of this work are the following; components borrowed from prior work are cited where they are used.

1. **A leakage audit of Elliptic++.** The official wallet features are shown to be lifetime summaries copied to every time step (constant for 100% of addresses that appear in more than one step; 10.6% of rows count steps that have not yet happened). Point-in-time address features are built from the raw transaction-address links as a leak-free replacement.
2. **Case-level review queues.** A flow-constrained grouping procedure (seeds, hop-limited expansion through high-risk links, hub filter, Louvain size cap) and a budget-matched queue protocol that extends the transaction-versus-actor comparison of Malik [8] to a third, case level.
3. **A benchmark under a strict protocol.** Five feature views and seven model variants with five seeds each, a wallet-level study, group proxy metrics against size-matched random groups, paired bootstrap tests, a label-scarcity study and a wider tuning grid.

4. **A counterfactual test of explanations.** Evidence edges selected by the gradient of the group score are deleted and compared with random, bottom-ranked and heuristic deletions.
5. **An open, tested implementation.** A documented Python package with 35 unit tests, the exact notebook sources, all result tables and figures [9]: https://github.com/Onethybeing/flare.

1.5 Scope

The study uses a public blockchain benchmark. Its labels are coarse (illicit, licit, unknown), it contains no confirmed laundering-ring identifiers, and it has no customer, device or merchant information. The results therefore concern research screening on Bitcoin data, not a production banking AML system, and all group-level quality measures are proxies based on known transaction labels.

1.6 Organisation of the report

Chapter 2 reviews related work and identifies the gap. Chapter 3 describes the Elliptic++ data and the leakage audit. Chapter 4 presents the methodology: features, models, group construction, scoring, queues and the evaluation protocol. Chapter 5 reports and discusses the results, and Chapter 6 concludes with limitations and the plan for the second stage of the project. Appendices list hyperparameters, additional results and the software.

Chapter 2

Literature Review

2.1 Machine learning for AML on public blockchains

Weber *et al.* [2] released the Elliptic dataset, a graph of 203,769 Bitcoin transactions over 49 time steps with 166 anonymised features, and showed that random forests on node features outperform graph convolutional networks in a temporal split, with a sharp performance drop after a dark-market shutdown. Elmougy and Liu [3] extended the data to Elliptic++, adding 822,942 wallet addresses, address features and transaction-address links, so that both transactions and actors (wallets) can be classified. Lorenz *et al.* [10] studied the same setting under label scarcity and found that active learning with few labels can approach fully supervised performance. More recently, the Elliptic2 dataset [11] introduced labelled subgraphs for money-laundering pattern detection, which is directly relevant to case-level evaluation.

2.2 Graph learning for fraud and anomaly detection

Graph neural networks (GNNs) learn node representations by aggregating information from neighbours [12]. Graph convolutional networks [13] use normalised spectral filters, while GraphSAGE [14] learns inductive aggregators that generalise to unseen nodes. Surveys of graph anomaly detection [15], [16] show that structure helps when anomalous entities are connected, but also that fraudsters camouflage themselves among benign neighbours; CARE-GNN [17] addresses this with relation-aware neighbour selection. Dynamic graphs are handled by snapshot models such as EvolveGCN [18] or continuous-time memories such as TGN [19].

2.3 Recent temporal and multi-view models on Elliptic

Several 2025-2026 studies build on these ideas. FG-EGCN [5] combines an EvolveGCN temporal encoder with a residual raw-feature branch and a node-wise gate. TDTW-GNN [6] samples direction-aware temporal tree walks and fuses in-flow and out-flow motifs. ST-CGNN [20] adds contrastive regularisation and an evidential uncertainty head for analyst triage. TWIN-GNN [21] encodes the Elliptic++ transaction and wallet graphs with separate GraphSAGE models fused by cross-graph attention; it reports that LightGBM remains stronger for transactions while fusion helps wallet-level detection, and acknowledges that its full-graph evaluation can see future neighbours. Related work

outside Bitcoin includes temporal heterogeneous contrastive learning for card fraud [22], edge-aware temporal message passing [23], event-driven dynamic graph construction [24] and adaptive temporal motifs (ATM-GAD, a preprint) [25]. Audit-HCL [26] explicitly targets collusive financial fraud with metapath-guided heterogeneous attention, a GRU temporal module and structural-temporal contrastive learning. These works improve node-level classification; they do not evaluate how alerts are grouped into cases under a review budget.

2.4 Granularity of review queues

Closest to this report, Malik [8] asks whether transaction-level and actor-level AML queues agree on Elliptic++. Using random forests at each level under a causal temporal protocol, the study finds low overlap between the queues and much higher illicit yield per review for the transaction queue than for the enriched address queue, and introduces budgeted metrics including case fragmentation. It does not use graph models or construct groups. The present work adds the case level, graph-based scoring and point-in-time cross-view features.

2.5 Explainability

GNNExplainer [27] learns a soft mask over edges and features that preserves a prediction; SubgraphX [28] searches subgraphs with Shapley values; SAGE-FIN [29] explains flagged nodes on Elliptic++ with Granger-causal attributions. A common critique is that attention weights or masks are not evidence of causal relevance unless tested by perturbation. This work therefore evaluates explanations by deletion: removing the selected edges must change the group score more than removing random edges.

2.6 Evaluation methodology

Leakage, the use of information unavailable at prediction time, is a well-documented source of over-optimistic results [7]. For rare positive classes, precision-recall analysis is more informative than ROC analysis [30], [31], and paired resampling [32] gives uncertainty estimates when the number of independent evaluation units (here, 14 test time steps) is small.

2.7 Research gap

Table 2.1 summarises the closest studies. None of them combines a leakage-audited, point-in-time protocol with budget-matched evaluation of case-level queues and a perturbation test of explanations; that combination is the focus of this report.

Table 2.1. Positioning of this work against the closest related studies.

Study	Data	Output unit	Temporal protocol	Budget evaluation	Explanation test
Weber et al. [2]	Elliptic	transaction	time split	no	no
Elmougy & Liu [3]	Elliptic++	tx, wallet	time split	no	no
FG-EGCN [5]	Elliptic	transaction	snapshots	no	no
ST-CGNN [20]	financial graphs	transaction	time split	fixed-budget P@K	no
TWIN-GNN [21]	Elliptic++	tx, wallet	full graph (transductive)	no	attention
Audit-HCL [26]	audit + synthetic	actor	dynamic graph	no	no
Malik [8]	Elliptic++	tx, actor queues	causal temporal	yes	no
This report	Elliptic++	**tx, wallet, case**	**as-of features, audited**	**yes, 4 budgets**	**deletion test**

Chapter 3

Data and Leakage Audit

3.1 The Elliptic++ dataset

Elliptic++ [3], [4] was obtained from a public Kaggle mirror that contains all eight official files. Table 3.1 lists them with the counts observed after loading. Transactions carry 93 local features, 72 neighbourhood-aggregate features (both anonymised by the original authors) and 17 augmented features such as input and output degree, BTC amounts and fees. Wallet addresses are linked to the transactions they spend into and receive from.

Table 3.1. Elliptic++ files and counts as loaded.

File	Rows	Content
txs_features.csv	203,769	txId, time step, 93 local + 72 aggregate + 17 augmented features
txs_classes.csv	203,769	4,545 illicit, 42,019 licit, 157,205 unknown
txs_edgelist.csv	234,355	directed money-flow edges between transactions
AddrTx_edgelist.csv	477,117	input address to transaction
TxAddr_edgelist.csv	837,124	transaction to output address
wallets_features.csv	1,268,260	address x time-step rows with 56 features
wallets_classes.csv	822,942	14,266 illicit, 251,088 licit, 557,588 unknown addresses
AddrAddr_edgelist.csv	2,868,964	address to address interactions

3.2 Temporal structure and labels

The data cover 49 time steps of roughly two weeks each. Figure 3.1 shows the number of transactions per step and its label composition. Unknown transactions dominate (77%); they are never treated as licit in this work. Using the chronological split of recent Elliptic++ studies [21], steps 1-30 form the training period (26,905 labelled transactions, 2,954 illicit), steps 31-35 the validation period (4,330; 690 illicit) and steps 36-49 the test period (15,329; 901 illicit).

The share of illicit transactions among labelled ones changes markedly over time (Figure 3.2). After step 43 the illicit share collapses, consistent with the dark-market shutdown documented by Weber *et al.* [2]. This concept drift affects every model and is analysed separately in Chapter 5.

3.3 Finding 1: all money-flow edges lie within a single time step

Every one of the 234,355 transaction edges connects two transactions of the same time step. Two consequences follow. First, a transaction's graph neighbourhood never contains future transactions, so full-graph and step-by-step (inductive) message passing coincide for the transaction graph; the transductive concern raised for multi-graph models [21] arises only through the address view. Second, flow groups are necessarily within-step objects; cross-step structure enters only through addresses that reappear over time.

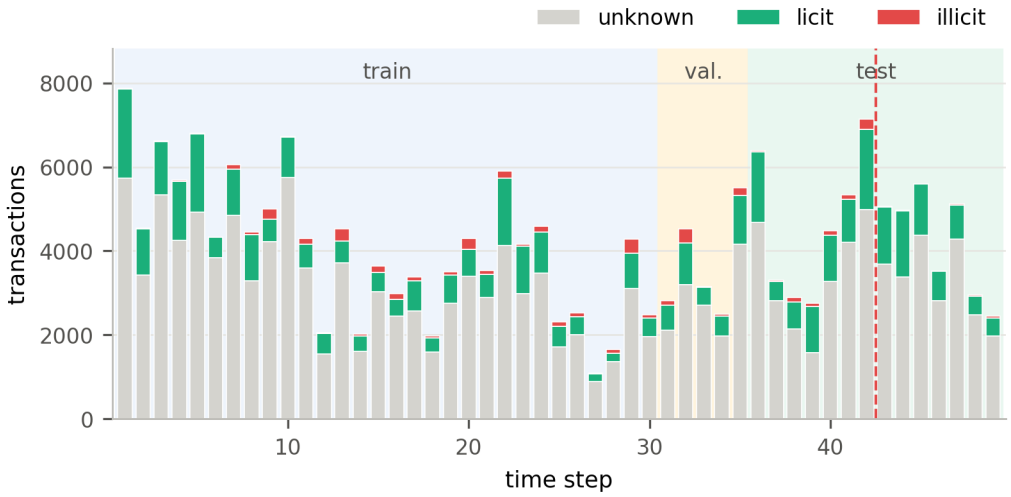


Figure 3.1. Transactions per time step by label. Shaded bands mark the training, validation and test periods; the dashed line marks the dark-market shutdown at step 43.

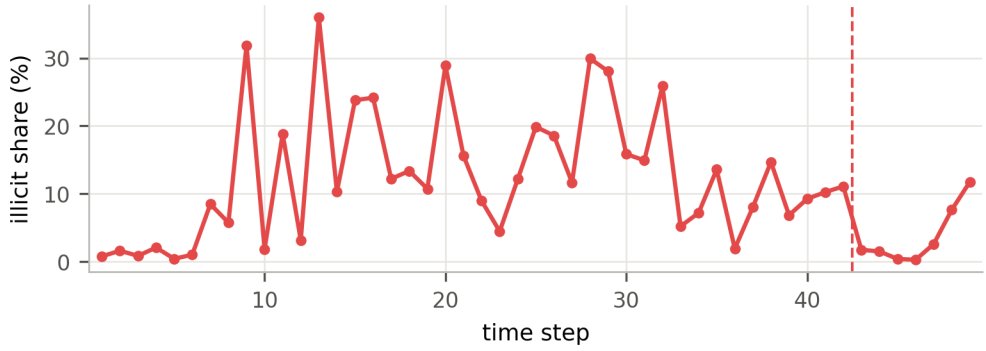


Figure 3.2. Illicit share of labelled transactions per time step.

3.4 Finding 2: the official wallet features are lifetime summaries

The wallet file has one row per address and time step, which suggests point-in-time values. A direct check shows otherwise (Table 3.2): for every address that appears in more than one step, the total number of transactions and the last block in which the address appears are identical in all of its rows, and in 10.6% of rows the reported number of active

time steps exceeds the number of steps observed so far. An address scored at step 25 therefore already carries information about its activity at step 45. In a chronological evaluation this is leakage in the sense of Kaufman *et al.* [7].

Table 3.2. Leakage audit of the official Elliptic++ wallet features.

Check	Result
Address-step rows matched to the transaction links	920,691
Addresses appearing in more than one time step	61,487
... with identical total_txs in every row	100%
... with identical last_block_appeared_in in every row	100%
Rows whose num_timesteps_appeared_in exceeds the steps observed so far	10.6%

3.5 Implications for the design

- Address features are rebuilt from the raw links using only the current and earlier steps (Section 4.3); the official features are kept only as an explicitly labelled comparator.
- Label history used as a feature is restricted to training-period labels from strictly earlier steps.
- Because no ring or campaign identifiers exist, group quality is measured with proxies (hit rate, purity, enrichment against random groups), always reported together with label coverage.

Chapter 4

Methodology

4.1 Overview

Figure 4.1 summarises the pipeline. Point-in-time features are computed from the raw data, risk models score each transaction, high-risk transactions are assembled into flow groups, and groups are reviewed in score order until the budget is exhausted. Every component is fitted on the training period, tuned on the validation period and evaluated once on the test period.

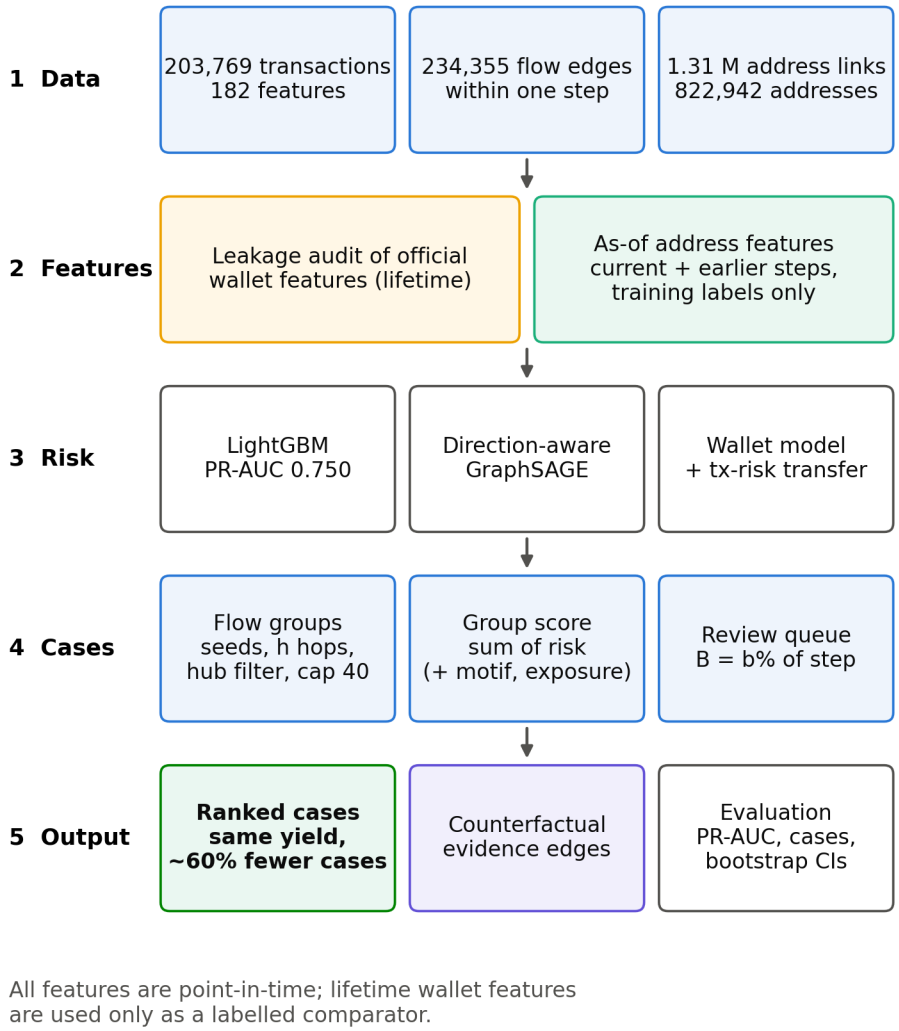


Figure 4.1. System pipeline. Solid arrows show the leakage-safe path used for all headline results; dashed red arrows show the lifetime-feature comparator.

4.2 Problem formulation

At time step t the observed data are the transactions V_t, their features x_i, the directed money-flow edges E_t and the address-transaction links. Labels y_i take values 1 (illicit), 0 (licit) or -1 (unknown). A risk model assigns a score r_i in [0, 1] to every transaction. With budget fraction b the analyst reviews

$$B_t = \lceil b \cdot |V_t| \rceil \tag{4.1}$$

transactions in step t. A queue is a rule that selects the reviewed set R_t with $|R_t| = B_t$. The aim is a queue that finds many illicit transactions while keeping R_t organised into few coherent cases.

4.3 Point-in-time address features

For an address a active at step t, current-step quantities $cur_k(a, t)$ (transactions sent and received, BTC volume, fees, number of transactions) are observable at the end of the step. History is accumulated over strictly earlier steps only,

$$\text{prior}_k(a,t) = \sum_{s<t} \text{cur}_k(a,s), \qquad \text{age}(a,t) = t - \min\{s : a \text{ active at } s\} \tag{4.2}$$

and label history uses only training-period labels from earlier steps, where T(a, s) is the set of transactions of address a in step s:

$$\text{prior_ill}(a,t) = \sum_{s<t,\ s\le 30} |\{\, i \in T(a,s) : y_i = 1 \,\}| \tag{4.3}$$

Fifteen such features (Appendix A) are averaged and maximised over each transaction's input addresses and output addresses, giving 60 cross-view columns (XV). The comparator XV-lifetime aggregates 13 official wallet features in the same way. Figure 4.2 contrasts the two constructions on the time axis.

4.4 Risk models

4.4.1 Tabular models

LightGBM [33] (gradient-boosted trees, learning rate 0.03, 63 leaves, early stopping on validation PR-AUC), random forests [34] (300 trees) and logistic regression with balanced class weights are trained on five feature views: local features (LF), all original features (AF), AF with the augmented features (AF+AUG), and AF+AUG with either the as-of cross-view features (XV) or the lifetime comparator. Models were implemented with

LightGBM, scikit-learn [35] and PyTorch [36].

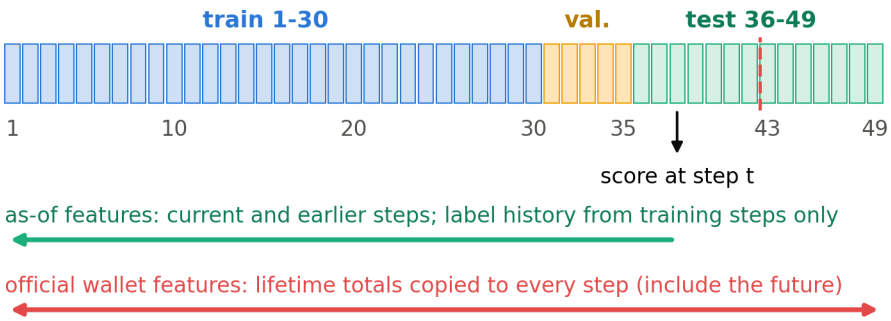


Figure 4.2. Chronological split and the difference between as-of features (current and earlier steps only) and the official lifetime wallet features (which include future activity).

4.4.2 Direction-aware GraphSAGE

Money flows have a direction: where funds came from and where they went carry different information. The graph model therefore extends GraphSAGE [14] with separate channels for incoming and outgoing neighbours and with edge weights w_e, which are used later for the counterfactual test:

$$h_v^{(l+1)} = \mathrm{ReLU}\left(W^{(l)} \left[h_v^{(l)} \,\|\, \frac{\sum_{u \to v} w_{uv}\, h_u^{(l)}}{\sum_{u \to v} w_{uv}} \,\|\, \frac{\sum_{v \to u} w_{vu}\, h_u^{(l)}}{\sum_{v \to u} w_{vu}} \right] \right) \tag{4.4}$$

Two layers of 128 units with dropout 0.3 are followed by a linear output. Training minimises a class-weighted binary cross-entropy on labelled training transactions only,

$$\mathcal{L} = -\frac{1}{|\mathcal{T}|} \sum_{i \in \mathcal{T}} \left[w_+ \, y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right], \qquad w_+ = \sqrt{n_- / n_+} \tag{4.5}$$

with early stopping on validation PR-AUC. Variants without direction (pooled neighbours) and without graph (an MLP) serve as ablations. All stochastic models use five seeds; the queue experiments use the seed-averaged score.

4.4.3 Wallet model

Wallet (address-step) rows are classified with LightGBM using either the as-of address features or the official lifetime features, each with and without *transaction-risk transfer*: the maximum and mean risk of the address's transactions in that step. To avoid in-sample leakage, the transaction risk used for training rows is produced out of fold (five folds over training steps).

4.5 Flow-constrained candidate groups

Groups are built separately for every time step on a graph that joins transactions by money flow (in either direction) and by shared addresses within the step. To avoid quadratic cliques around busy addresses, the transactions of an address-step pair are chained in id order and only pairs with at most ten transactions are linked. The expansion threshold of step t is the (1-q) quantile of the risk scores,

$$\tau_t = Q_{1-q}(\{\, r_v : v \in V_t \,\}) \tag{4.6}$$

and the procedure is given in the box below and illustrated in Figure 4.3.

Procedure 1: flow-constrained groups for step t
Input: step graph, risk r, seed fraction σ, quantile q, hop limit h, hub degree H = 30, size cap M = 40.
1. Seeds S_t: the max(1, round(σ $|V_t|$)) highest-risk transactions; σ = sfm × b scales with the budget.
2. For each seed whose degree is at most H: breadth-first expansion for at most h hops, adding a neighbour u only if $r_u \geq \tau_t$ and deg(u) ≤ H. Hubs are never expanded through.
3. Take connected components of the collected nodes; keep those containing a seed.
4. Split components larger than M with Louvain community detection [37]; keep pieces that contain a seed.
Output: candidate groups (a seed with no qualifying neighbour forms a singleton group).

Two baselines are compared. *Connected components* takes all transactions above τ_t together with the seeds and returns the components that contain a seed, with no hop limit, hub filter or size cap. The *1-hop neighbourhood* baseline returns each seed together with all of its neighbours.

4.6 Group scoring

Each group is described by its size n, the maximum, mean, top-3 mean and sum of member risks, a flow-motif density and a known-bad exposure. The motif density counts members that split funds (out-degree at least two), merge funds (in-degree at least two) or pass funds through (in-degree and out-degree one) inside the group's flow sub-graph:

$$\mathrm{motif}(G) = \tfrac{1}{n}\left(|\{v : d_G^{out}(v) \geq 2\}| + |\{v : d_G^{in}(v) \geq 2\}| + |\{v : d_G^{in}(v) = d_G^{out}(v) = 1\}|\right) \tag{4.7}$$

Exposure is the mean, over members, of an indicator that one of the member's addresses had an illicit training label in an earlier step. The proposed score combines these terms,

$$S(G) = \mathrm{base}(G) \cdot (1 + \beta\,\mathrm{motif}(G) + \gamma\,\mathrm{exposure}(G)) \cdot n^{-\delta}, \qquad \mathrm{base} \in \{\mathrm{sum}, \mathrm{top3}\} \tag{4.8}$$

where $\delta = 0$ behaves like a sum and $\delta = 1$ turns a sum base into a mean. The weights are searched on the validation period over β in $\{0, 0.25, 0.5, 1\}$, γ in $\{0, 0.5, 1\}$ and δ in $\{0, 0.25, 0.5\}$, jointly with the seed multiplier (1.0, 1.5), the quantile q (0.05, 0.1, 0.2) and the hop limit h (1, 2), maximising illicit transactions found summed over the four budgets. The plain max, mean, top-3 and sum scores are reported as ablations.

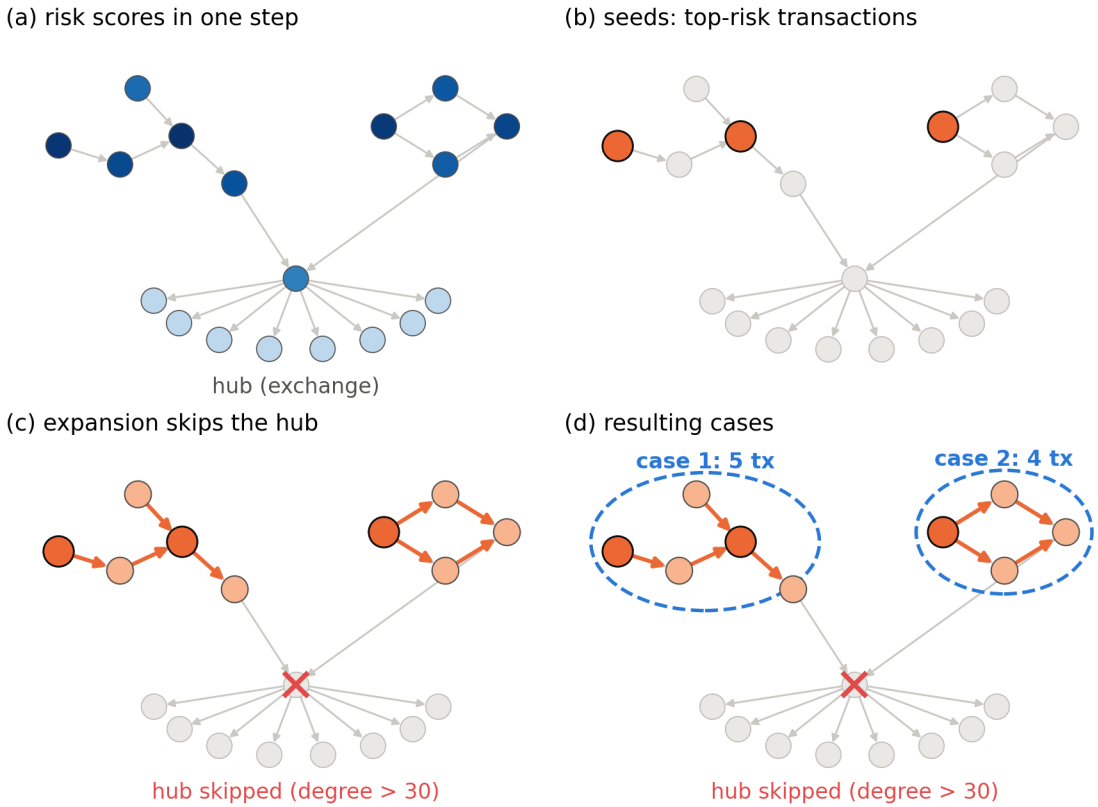


Figure 4.3. Group construction on a toy step: (a) risk scores, (b) seeds, (c) hop-limited expansion through high-risk links that skips the hub, (d) resulting cases. Connected components would merge both flows through the hub.

4.7 Review queues and evaluation metrics

A group queue reviews groups whole in score order; the group that would overflow the budget is truncated to its highest-risk members, and any remaining capacity is filled with the highest-risk unreviewed transactions, so that every queue reviews exactly B_t transactions. The transaction queue is the special case without groups. The wallet queue reviews addresses in wallet-model order, each covering its transactions in the step. Figure 4.4 illustrates the difference between the transaction and group queues.

Queues are compared on illicit transactions found, precision among labelled reviewed transactions, recall, label coverage and fragmentation. Fragmentation counts the connected

components c(R_t) of the reviewed set in the step graph:

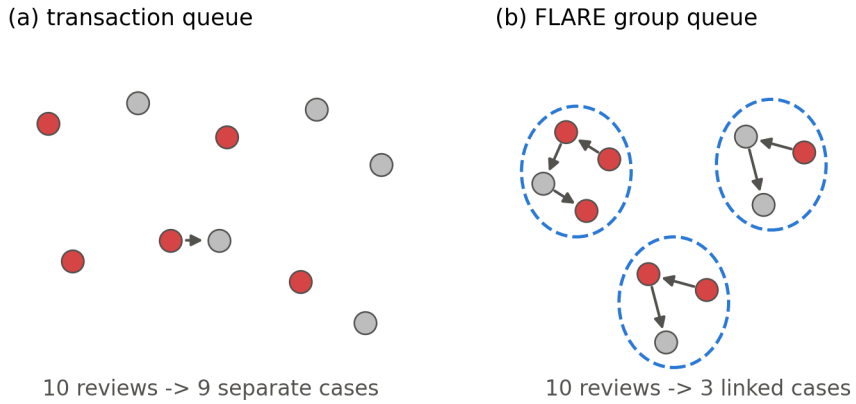


Figure 4.4. Transaction queue versus group queue for the same budget of ten reviews.

$$\text{cases per } 100 = 100 \cdot \frac{\sum_t c(R_t)}{\sum_t |R_t|}, \qquad \text{illicit per case} = \frac{\sum_t |\{i \in R_t : y_i = 1\}|}{\sum_t c(R_t)} \tag{4.9}$$

Node-level models are compared with PR-AUC (primary [30]), ROC-AUC, illicit F1 at the threshold that is optimal on validation, P@100 and P@500, and F1 before and after the step-43 shutdown. Group quality is measured on the ten highest-scoring groups per test step by the share containing at least one (lenient) or two (strict) illicit members, the illicit share among labelled members, label coverage, and enrichment relative to size-matched random connected groups grown by breadth-first search from random transactions of the same step (twenty per group).

4.8 Counterfactual evidence test

For the top groups of the GraphSAGE pipeline, all flow edges of the step receive weight one. The gradient of the group score with respect to the edge weights ranks candidate evidence edges. Deleting the top-m edges gives the drop

$$\Delta_{top}(G) = S(G;\, \mathbf{w}) - S\big(G;\, \mathbf{w} \odot (\mathbf{1} - \mathbf{1}_{E_m^*})\big) \tag{4.10}$$

which is compared with deleting m random candidate edges (ten draws), the m lowest-ranked edges, and the m edges that touch the highest-risk member (a heuristic an analyst might use). Sufficiency keeps only the top-m edges inside the receptive field.

4.9 Experimental protocol

Differences between queues are tested with a paired bootstrap over the 14 test steps [32] (2,000 resamples); for queues a and b with per-step counts a_t and b_t, the statistic is the total difference $\Sigma_t(a_t - b_t)$. Model metrics are reported as mean and standard deviation over seeds. All experiments ran as Kaggle notebooks on an NVIDIA T4 GPU (main run 30.9 minutes, supplementary run 11.2 minutes); groups and graphs use NetworkX [38]. Table 4.1 lists the experiments.

Table 4.1. Experiments and the research questions they address.

ID	Experiment	Question
E1	Transaction models: 5 feature views, LR, RF, LightGBM, MLP, GraphSAGE (directed, undirected)	RQ1
E2	Wallet models: as-of vs lifetime features, with and without transaction-risk transfer	RQ2
E3	Review queues at 0.5%, 1%, 2%, 5% budgets; ablations of score, hub filter and size cap	RQ3
E4	Group proxy metrics against size-matched random groups	RQ3
E5	Counterfactual deletion test of evidence edges	RQ5
E6	Per-step curves and paired bootstrap confidence intervals	RQ1, RQ3
X1	Wider tuning grid (seed multiplier up to 3.0)	RQ4
X2	Label scarcity: 100%, 50%, 20%, 10% of training labels (3 subsets each)	RQ4

Chapter 5

Results and Discussion

5.1 Transaction-level detection (RQ1)

Table 5.1 reports test-period results averaged over seeds. LightGBM with the as-of cross-view features is the best model (PR-AUC 0.750 ± 0.001), followed closely by LightGBM without them (0.747) and with the lifetime comparator (0.745). Random forests are about 0.02 lower. Tree models therefore remain the strongest transaction scorers, in line with the original Elliptic study [2] and TWIN-GNN [21].

Table 5.1. Transaction-level results on test steps 36-49 (mean over seeds, with standard deviation for PR-AUC; five seeds for LightGBM and GNNs, three for random forests, one for logistic regression).

Model	Features	PR-AUC	ROC-AUC	Illicit F1	P@100	F1 (36-42)	F1 (43-49)
LightGBM	AF+AUG+XV	0.750 ± 0.001	0.924	0.764	1.00	0.875	0.029
LightGBM	AF+AUG	0.747 ± 0.002	0.919	0.757	1.00	0.875	0.029
LightGBM	AF+AUG+XV-lifetime	0.745 ± 0.002	0.916	0.751	1.00	0.874	0.027
LightGBM	AF	0.745 ± 0.001	0.914	0.762	1.00	0.879	0.028
Random forest	AF+AUG+XV	0.727 ± 0.003	0.898	0.773	1.00	0.877	0.041
LightGBM	LF	0.726 ± 0.001	0.900	0.728	1.00	0.837	0.022
Random forest	AF	0.725 ± 0.001	0.884	0.700	1.00	0.842	0.020
GraphSAGE (dir.)	AF+AUG+XV	0.720 ± 0.022	0.938	0.680	1.00	0.788	0.032
GraphSAGE (undir.)	AF+AUG	0.561 ± 0.043	0.895	0.537	0.96	0.633	0.013
MLP	AF+AUG	0.486 ± 0.045	0.893	0.530	0.78	0.615	0.018
Logistic regr.	AF+AUG+XV	0.402	0.881	0.451	0.57	0.606	0.037
GraphSAGE (dir.)	AF+AUG	0.399 ± 0.056	0.888	0.453	0.74	0.555	0.012

The graph model benefits strongly from the cross-view features: directed GraphSAGE improves from 0.399 to 0.720 PR-AUC when the as-of address features are added, whereas LightGBM gains only 0.003. Without them, the undirected variant (0.562) is better than the directed one (0.399), which suggests that splitting the already sparse neighbourhoods into two channels hurts when node features are weak. Figure 5.1 compares all variants.

Figure 5.2 shows illicit F1 per test step. Before step 43 the best models reach F1 between 0.75 and 0.95; from step 43 onwards every model collapses to almost zero. This reproduces the dark-market shutdown effect [2]: the illicit behaviour that the models

learned largely disappears, and the few remaining illicit transactions look different. No feature view or architecture escapes this drift, which is a limitation of supervised training on a fixed historical period rather than of any specific model.

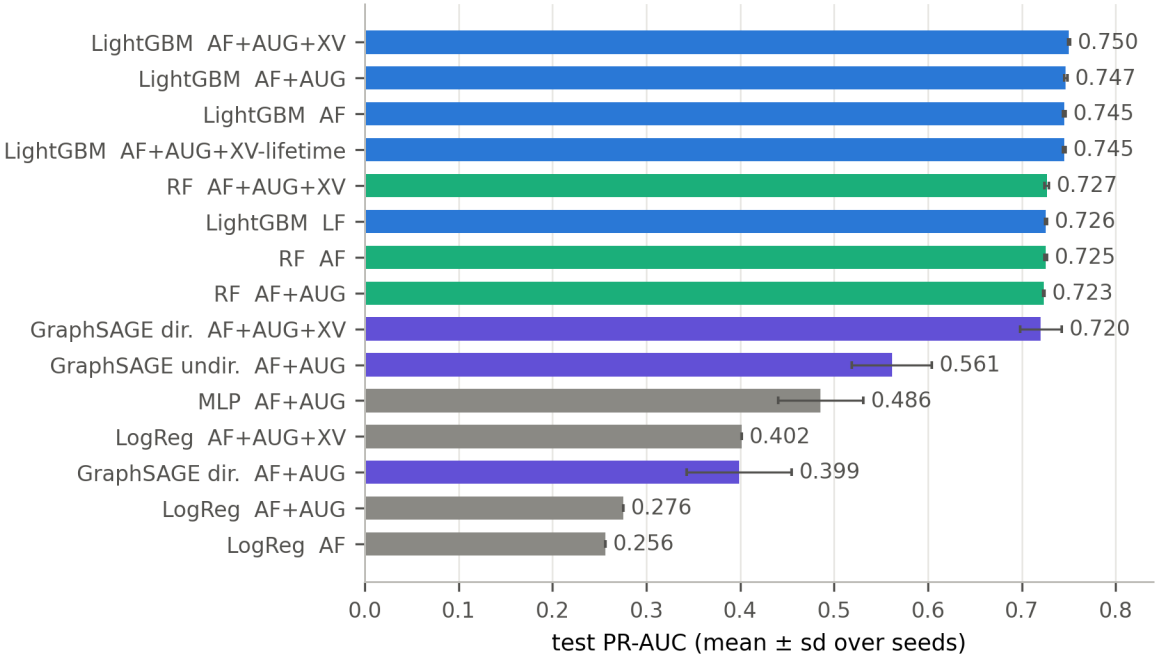


Figure 5.1. Test PR-AUC of all model and feature-view combinations (mean ± standard deviation over seeds).

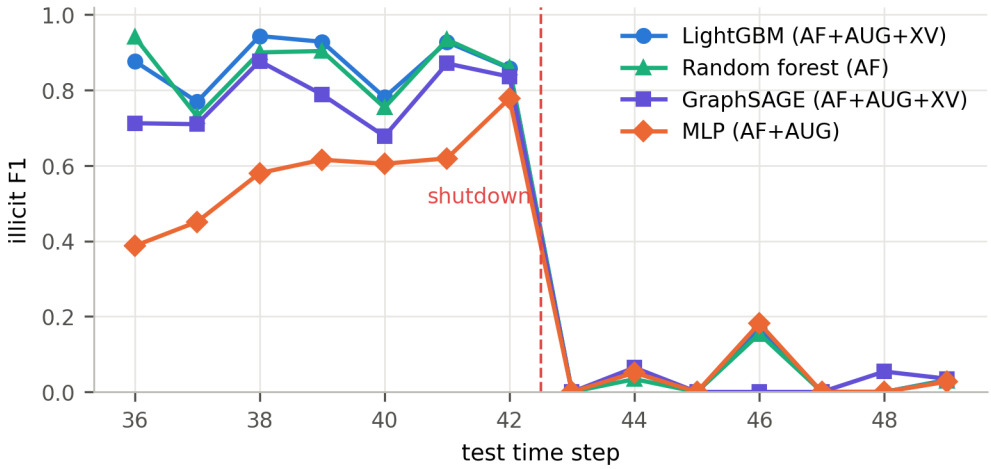


Figure 5.2. Illicit F1 per test time step (seed 0). All models fail after the dark-market shutdown at step 43.

The most important LightGBM features (by gain) are the transaction size, several local features, and two as-of address features: the mean illicit history of output addresses and the illicit fraction of input addresses. Address history therefore carries real signal even when restricted to past training labels.

5.2 Cross-view features and leakage (RQ2)

Table 5.2 and Figure 5.3 compare point-in-time and lifetime features. At the transaction level the lifetime comparator is slightly worse than the as-of view (0.745 versus 0.750). At the wallet level, the lifetime features look better *on their own* (address PR-AUC 0.345 versus 0.273), which is expected because they summarise each address's entire future activity. Once transaction risk is transferred to wallets, however, the as-of model is best (0.645 versus 0.636): the leaked future information adds nothing that the transaction model does not already capture.

Table 5.2. Wallet-level results on test steps (three seeds). New-address PR-AUC covers addresses first seen in the test period (87,279 addresses).

Wallet features	Row PR-AUC	Address PR-AUC	New-address PR-AUC	Row F1
As-of	0.271 ± 0.029	0.273	0.268	0.339
Official (lifetime)	0.340 ± 0.017	0.345	0.343	0.375
As-of + transaction risk	0.638 ± 0.002	0.644	0.644	0.627
Lifetime + transaction risk	0.628 ± 0.004	0.636	0.638	0.638

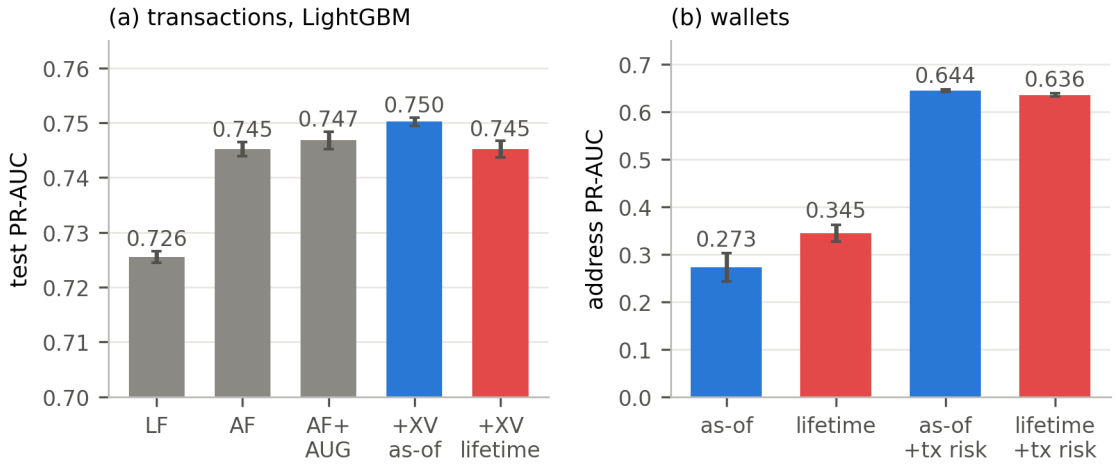


Figure 5.3. (a) LightGBM PR-AUC by transaction feature view. (b) Wallet-level PR-AUC with as-of and lifetime features, with and without transaction-risk transfer.

Transaction-risk transfer is the key cross-view signal: it raises wallet-level PR-AUC from 0.27 to 0.65. Combined with the large gain of GraphSAGE from address features, this supports the view that the transaction and address views are complementary, while showing that the complementarity can be exploited without future information.

5.3 Case-level review queues (RQ3)

Table 5.3 is the central result. With LightGBM scores and a 1% budget, the FLARE queue finds 210 illicit transactions, the transaction queue 208. The yields are statistically indistinguishable, but the group queue needs 36.4 separate cases per 100 reviews instead of 83.7, more than doubling the number of illicit transactions per case (0.92 versus 0.40).

Table 5.3. Review queues on test steps 36-49 at a 1% budget per step. Precision is computed among labelled reviewed transactions; unknown transactions are not counted as licit.

Risk	Queue	Illicit found	Precision	Recall	Cases / 100	Illicit / case
LightGBM	Transaction queue	208	0.986	0.231	83.7	0.40
LightGBM	Wallet queue	96	0.478	0.107	31.0	0.49
LightGBM	**FLARE groups (ours)**	210	0.981	0.233	36.4	0.92
LightGBM	Connected components	205	0.958	0.228	32.4	1.01
LightGBM	1-hop neighbourhood	167	0.773	0.185	26.7	1.00
GraphSAGE	Transaction queue	146	0.948	0.162	90.9	0.26
GraphSAGE	Wallet queue	95	0.475	0.105	31.0	0.49
GraphSAGE	**FLARE groups (ours)**	174	0.916	0.193	45.8	0.61
GraphSAGE	Connected components	181	0.919	0.201	42.0	0.69
GraphSAGE	1-hop neighbourhood	158	0.760	0.175	28.6	0.88

The wallet queue performs worst (96 illicit transactions at 1%), in line with the low actor-level yield reported by Malik [8]. The 1-hop neighbourhood baseline is the least fragmented but pulls in many unrelated neighbours and loses yield (167). Connected components behave much like flow groups. Figure 5.4 shows the same pattern across all budgets for LightGBM scores and Figure 5.5 for GraphSAGE scores.

Figure 5.6 summarises the paired bootstrap intervals for the difference between the FLARE queue and the transaction queue. With LightGBM risk, the difference in illicit found is -7 [-24, 10] at 0.5%, +2 [-19, 26] at 1%, +12 [-23, 46] at 2% and +20 [3, 41] at 5%, while the number of cases falls by 296 [212, 400] at 1% and by 769 at 5%. With GraphSAGE risk the group queue finds significantly more illicit transactions at 0.5% (+29 [2, 60]), 2% (+44 [3, 90]) and 5% (+22 [1, 44]). The weaker the transaction scorer, the more the graph neighbourhood of high-risk seeds adds.

Table 5.4 isolates the components of the method at the 1% budget. Mean-based scores perform badly because they favour singletons, so the queue degenerates into a fragmented transaction queue with lower yield. The tuned score and the plain sum are close (210 versus 202 illicit with LightGBM; 174 versus 177 with GraphSAGE), so the extra motif and exposure terms do not provide a reliable gain on the test period. Removing the hub

filter or the size cap changes little because most groups are small (median size two to three).

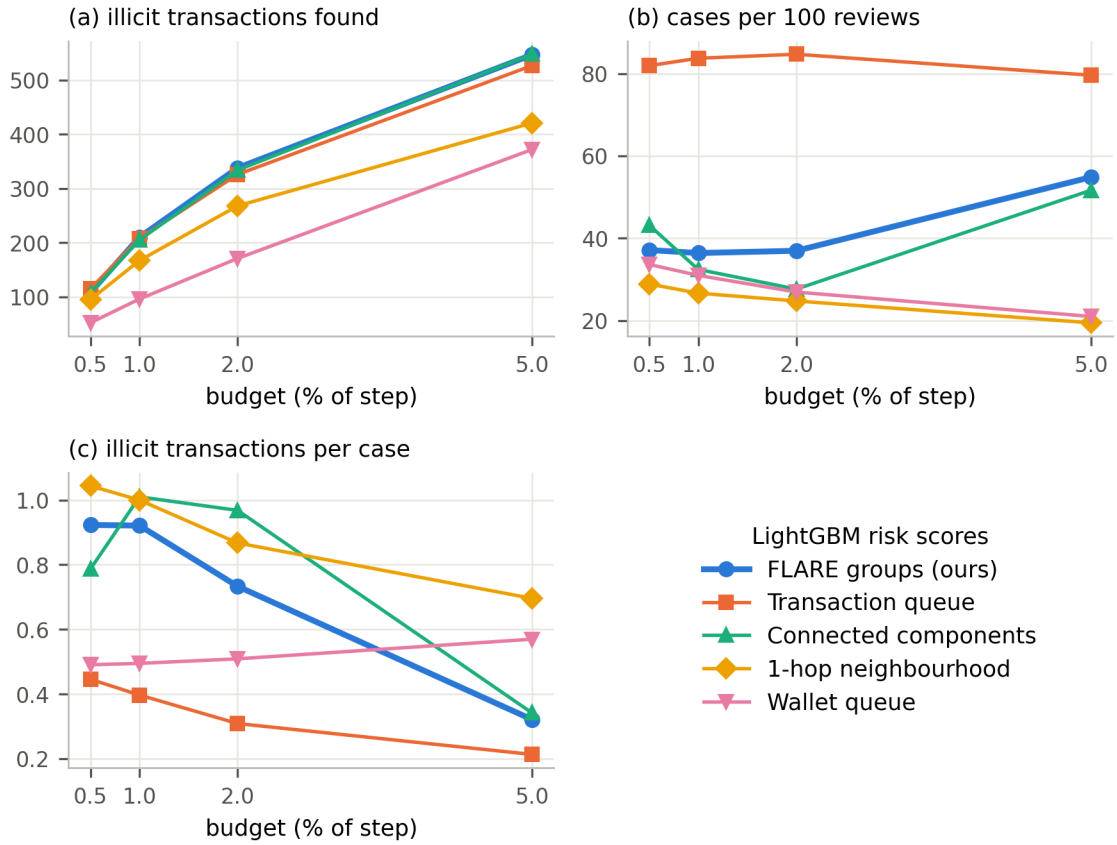


Figure 5.4. Review queues versus budget with LightGBM risk: illicit transactions found, cases per 100 reviews and illicit transactions per case.

5.4 Group-level proxy quality

Table 5.5 evaluates the ten highest-scoring groups of each test step. Around 38% of top flow groups contain at least one known-illicit transaction and about one third contain at least two. Among labelled members, 96% are illicit with LightGBM risk, against about 5% in size-matched random connected groups: an enrichment of 17 times. All three grouping methods are strongly enriched (Figure 5.7); flow groups stay smaller than connected components (maximum size 21 versus 49).

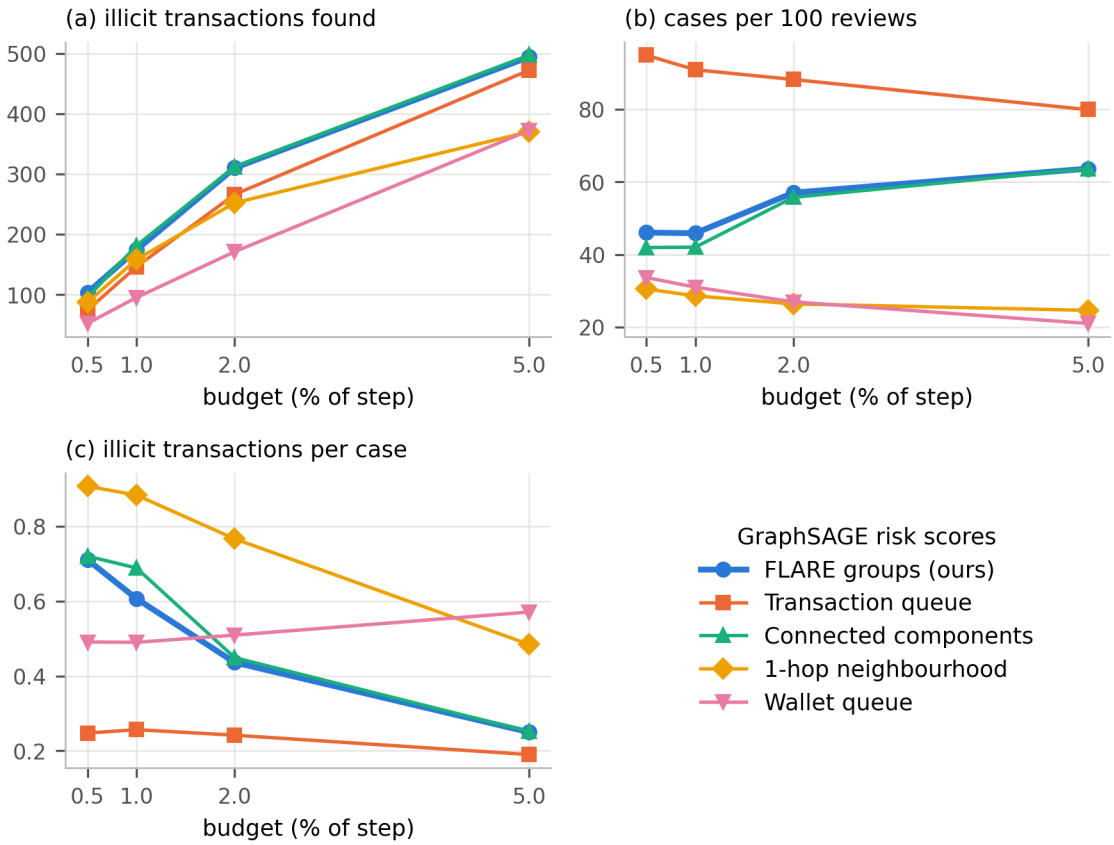


Figure 5.5. Review queues versus budget with GraphSAGE risk.

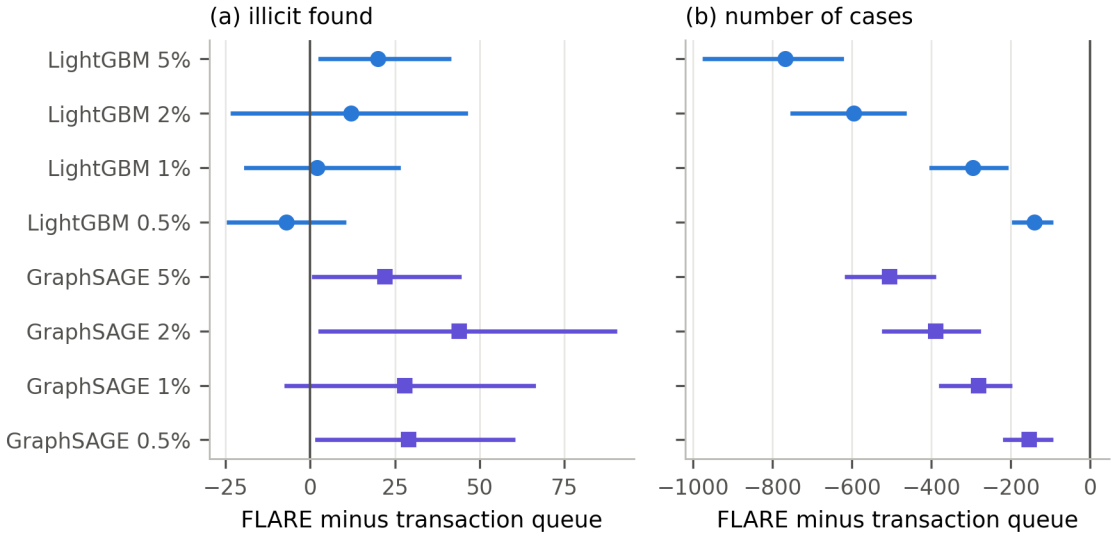


Figure 5.6. FLARE minus transaction queue: total difference over the 14 test steps with 95% paired-bootstrap intervals, for illicit found (left) and number of cases (right).

5.5 Robustness: label scarcity and tuning (RQ4)

Annotating transactions is expensive, so realistic AML settings have few labels [10]. In the label-scarcity experiment the training labels (and the label history used by the as-of features) were subsampled per class to 50%, 20% and 10%, with three random subsets each and the group configuration frozen from the main run (Table 5.6, Figure 5.8). Transaction PR-AUC decreases only slightly, from 0.750 to 0.722, but the yield of the transaction queue at the 1% budget falls by about 16% (from 208 to 170-175), because the top of the ranking degrades first. The group queues keep their yield at about 200-212 illicit transactions, since the graph neighbourhood of the seeds recovers illicit transactions that the weaker model ranks lower. With 10-20% of the labels, groups therefore find 15-17% more illicit transactions, and they still need about 60% fewer cases.

Table 5.4. Ablations of the FLARE queue at a 1% budget.

Variant (FLARE groups)	LightGBM: illicit	LightGBM: cases / 100	GraphSAGE: illicit	GraphSAGE: cases / 100
Proposed score (tuned)	210	36.4	174	45.8
Sum of risk	202	35.1	177	45.7
Top-3 mean	206	59.4	161	72.5
Maximum	208	48.6	160	63.9
Mean	160	83.9	121	90.1
No hub filter	210	35.6	177	44.9
No size cap	209	35.9	174	45.8

Table 5.5. Proxy quality of the top-10 groups per test step (140 groups per method). Purity is the illicit share among labelled members; coverage is the labelled share of members.

Risk	Method	Mean size	Max size	Hit ≥ 1	Hit ≥ 2	Purity	Cover.	Random purity	Enrich.
LightGBM	FLARE (ours)	4.2	21	0.39	0.33	0.96	0.43	0.055	17.3x
LightGBM	conn. comp.	4.8	49	0.38	0.32	0.96	0.44	0.048	19.8x
LightGBM	1-hop	3.0	6	0.43	0.31	0.90	0.38	0.047	19.0x
GraphSAGE	FLARE (ours)	3.4	36	0.29	0.21	0.92	0.37	0.051	18.1x
GraphSAGE	conn. comp.	3.6	36	0.31	0.22	0.93	0.38	0.057	16.1x
GraphSAGE	1-hop	2.9	14	0.39	0.27	0.83	0.37	0.049	17.2x

In the main run the selected seed multiplier (1.5) lay at the edge of its grid. Widening the grid to {1.0, 1.5, 2.0, 3.0} moves the optimum inside it (2.0 for LightGBM, 1.5 for GraphSAGE) and leaves the conclusions unchanged: at 1%, flow groups find 209 illicit transactions with LightGBM risk (transaction queue 208) and 171 with GraphSAGE risk (transaction queue 147). The re-tuned extra score terms, however, gave 64.9 cases per 100

reviews against 41.2 for the plain sum score, confirming that weights tuned on five validation steps do not transfer reliably. A plain sum-of-risk group score is the robust recommendation.

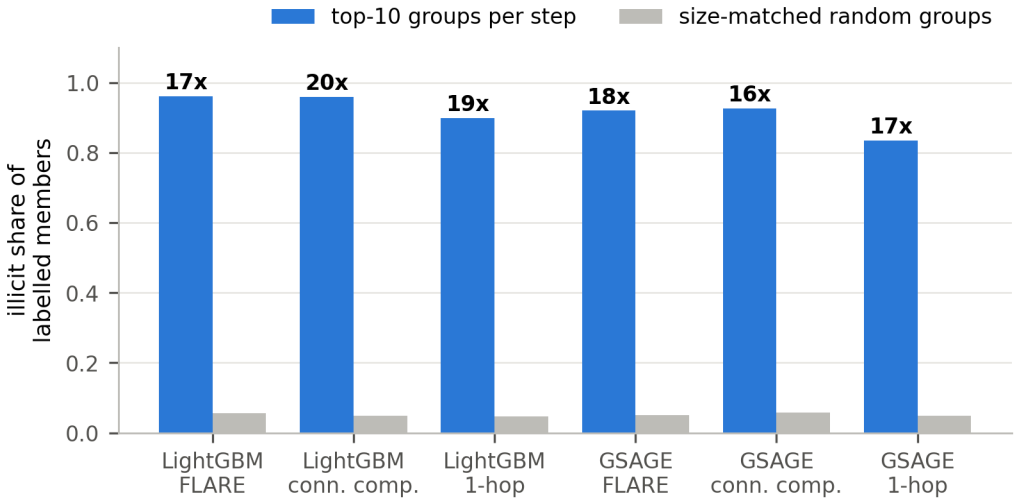


Figure 5.7. Illicit share of labelled members in top groups versus size-matched random groups; labels give the enrichment factor.

Table 5.6. Label scarcity (LightGBM risk; mean ± standard deviation over three label subsets).

Training labels	Tx PR-AUC	Illicit @1%: tx queue	Illicit @1%: FLARE	Illicit @1%: conn. comp.	Cases/100: tx queue	Cases/100: FLARE
100% (26,905)	0.750	207.7 ± 3.1	207.3 ± 5.0	203.3	83.3	35.2
50% (13,452)	0.753	193.3 ± 14.0	212.0 ± 5.0	210.7	84.3	34.9
20% (5,381)	0.732	170.3 ± 9.0	199.7 ± 5.9	203.3	87.0	36.5
10% (2,690)	0.722	175.3 ± 11.0	201.3 ± 11.5	203.3	87.1	33.0

5.6 Counterfactual evidence (RQ5)

Table 5.7 and Figure 5.9 report the deletion test on the top GraphSAGE groups. Deleting the single highest-gradient edge lowers the group score by 0.256 on average, against 0.069 for a random edge and 0.029 for the lowest-ranked edge. With three and five deleted edges the gradient selection (0.600 and 0.780) clearly beats both random deletion (0.276 and 0.404) and the heuristic of removing edges around the highest-risk member (0.429 and 0.445); for a single edge the heuristic is slightly stronger (0.305). The selected edges therefore carry genuine evidence. The test also has limits: the top edge beats a random edge in only about 60% of groups, and keeping only the selected edges retains 87-95% of the score, indicating that much of the score comes from the members' own features.

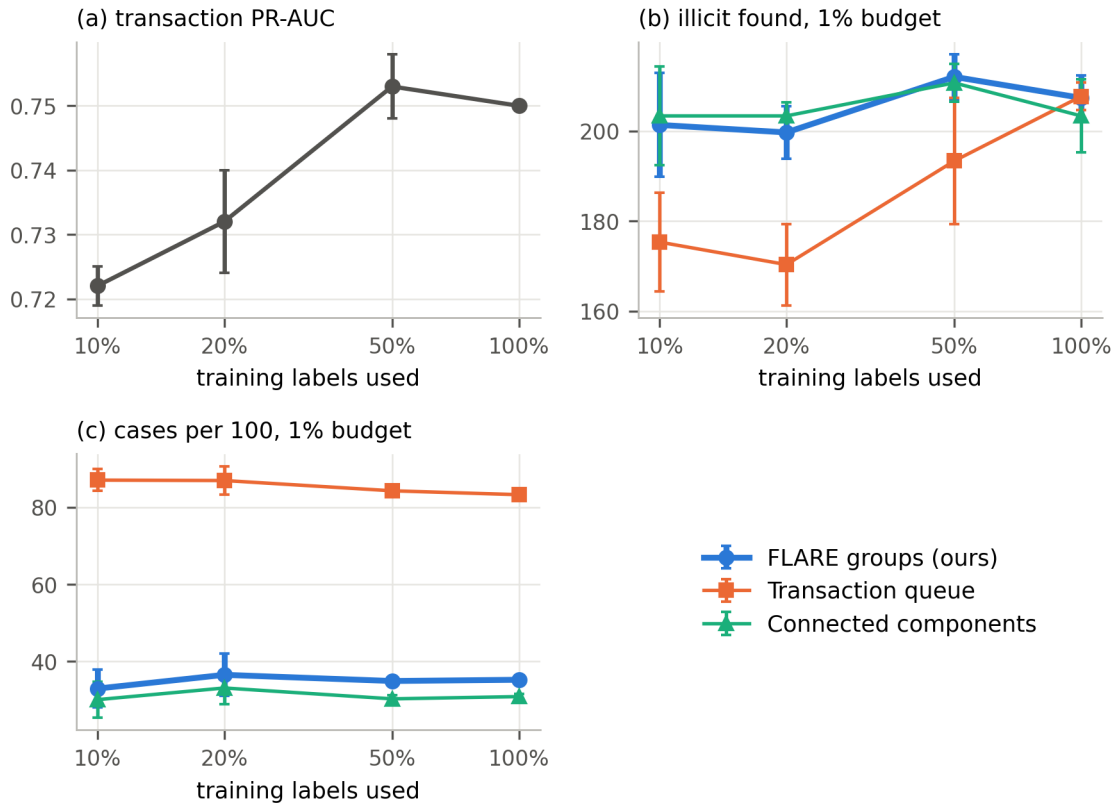


Figure 5.8. Label scarcity: transaction PR-AUC (left), illicit found (centre) and cases per 100 reviews (right) at a 1% budget.

Table 5.7. Counterfactual deletion test on the top GraphSAGE flow groups (score drop after deleting m edges).

Edges deleted (m)	Groups	Drop: gradient-top	Drop: heuristic	Drop: random	Drop: gradien t-bottom	Top > random	Sufficiency
1	117	0.256	0.305	0.069	0.029	59%	0.87
3	106	0.600	0.429	0.276	0.170	62%	0.94
5	84	0.779	0.445	0.404	0.312	62%	0.95

One property of the method should be noted. With normalised mean aggregation, an edge into a node that has only one neighbour in that direction has zero gradient, because its weight cancels in the mean; such edges are never selected even though deleting them changes the prediction. The released implementation documents this with a unit test.

5.7 Case study

Figure 5.10 shows one flow group from test step 37 selected for readability (6-14 members with money-flow edges). It links three known-illicit transactions through nine unlabelled ones by chains, a merge and shared addresses. A transaction queue would present these as

separate alerts, and the unlabelled members, several with risk above 0.9, might be reviewed without the context that connects them to confirmed illicit activity. Presented as one case, the flow structure itself becomes part of the evidence.

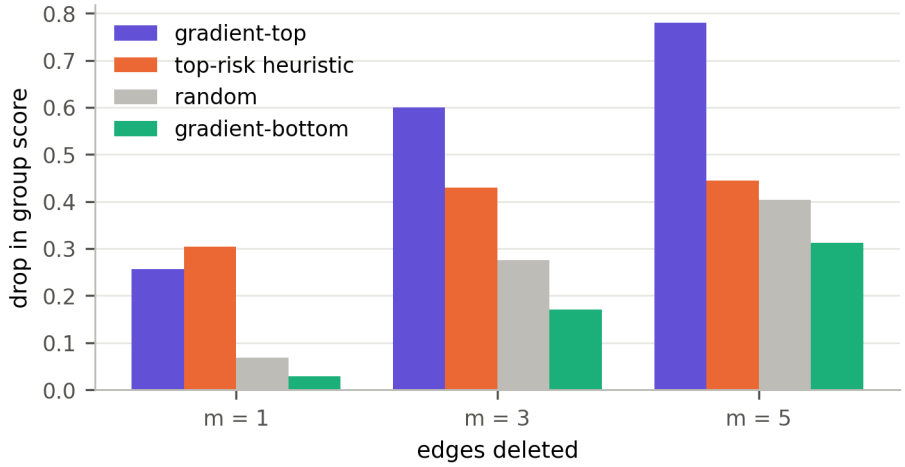


Figure 5.9. Drop in group score after deleting m evidence edges selected by gradient, by the top-risk-member heuristic, at random, or from the bottom of the gradient ranking.

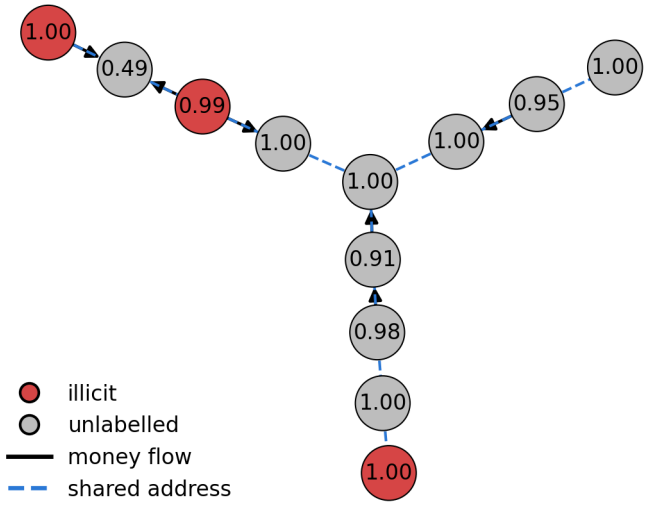


Figure 5.10. A flow group from test step 37: twelve transactions, three known illicit (red) and nine unlabelled (grey). Solid arrows are money flows, dashed lines shared addresses; labels are LightGBM risk.

5.8 Discussion

5.8.1 Economic interpretation

The results can be read as a statement about investigation productivity. At equal review volume, the group queue delivers the same number of confirmed illicit transactions in

fewer than half as many cases. If the cost of an investigation has a fixed component per case (opening a file, gathering context, writing a report) in addition to a per-transaction component, the cost per detected illicit transaction falls accordingly. In settings with few labels, which are the norm for new typologies, the group queue additionally finds more illicit activity for the same budget. Both effects matter for regulators and compliance units that must justify their resource allocation.

5.8.2 Why grouping helps

Grouping exploits two facts visible in the data: illicit transactions tend to be linked by money flows and shared addresses within the same time step, and a model's mistakes near the top of the ranking are partly recovered by looking at the neighbours of confident predictions. When the scorer is strong and labels are plentiful (LightGBM with all labels), the second effect is small and grouping mainly reorganises the same detections into cases. When the scorer is weaker (GraphSAGE) or trained on fewer labels, the neighbourhood adds detections.

5.8.3 Negative results and threats to validity

- The proposed motif and exposure terms do not improve on a plain sum of member risk on the test period, and connected components of the risk-thresholded graph are an equally strong grouping baseline in yield. The contribution lies in case-level evaluation and in the hop, hub and size constraints that keep cases compact, not in a new score.
- Group quality is measured with proxies because Elliptic++ has no laundering-ring identifiers; unknown labels dominate.
- There are only 14 test steps and small per-step budgets (about 16 to 70 transactions at 1%), so some differences have wide intervals; paired bootstrap tests were used throughout.
- All models fail after step 43. The case-level results therefore depend mainly on steps 36-42, where detections exist.
- Group parameters and score weights were tuned on five validation steps, which is a small basis for weight selection.

Chapter 6

Conclusions and Future Scope

6.1 Summary of findings

1. **RQ1.** Under a strict chronological protocol LightGBM is the strongest transaction scorer (PR-AUC 0.750). Point-in-time address features help a graph model substantially (GraphSAGE 0.399 to 0.720) and LightGBM marginally. Every model fails after the step-43 dark-market shutdown.
2. **RQ2.** The official Elliptic++ wallet features are lifetime summaries that leak future information. Leak-free replacements built from the raw links perform as well or better once transaction risk is transferred to wallets (address PR-AUC 0.645 versus 0.636).
3. **RQ3.** Case-level queues find as many illicit transactions as transaction queues at every budget while reducing the number of separate cases by about 55-60%; with GraphSAGE scores they also find significantly more illicit transactions at most budgets. Top groups are 16-20 times more illicit-enriched than random groups of the same size.
4. **RQ4.** With 10-20% of the training labels, group queues find 15-17% more illicit transactions than transaction queues. The conclusions are stable under a wider tuning grid, but tuned score weights are not; a plain sum-of-risk group score is recommended.
5. **RQ5.** Gradient-selected evidence edges lower group scores two to four times more than random edges when deleted, so they carry real evidence, with documented limits.

6.2 Limitations

The study is confined to a public Bitcoin benchmark with coarse labels and no ring identifiers, which limits group-level evaluation to proxies and limits transfer to bank AML. Flow edges never cross time steps in this data, so the groups are within-step objects; laundering that unfolds over weeks is captured only indirectly through addresses. The counterfactual test explains the GraphSAGE pipeline and not the LightGBM scores used in most queue results.

6.3 Future scope (plan for BTP-2)

- **Validated group labels.** Evaluate case-level queues on Elliptic2 [11], which provides labelled money-laundering subgraphs, to replace proxy metrics with direct group precision and recall.
- **Cross-step cases.** Link groups across time steps through recurring addresses to follow flows that span several weeks, and evaluate time-to-detection.
- **Cost-aware budgets.** Model investigation cost explicitly (a fixed cost per case plus a cost per transaction) and choose group sizes and budgets that minimise expected cost per detected illicit transaction.
- **Drift adaptation.** Study periodic retraining and unsupervised signals to soften the collapse after structural breaks such as the step-43 shutdown.
- **Analyst-facing evaluation.** Build the case-review console sketched in Figure 6.1 and measure, with domain users, whether case-level presentation shortens investigation time.

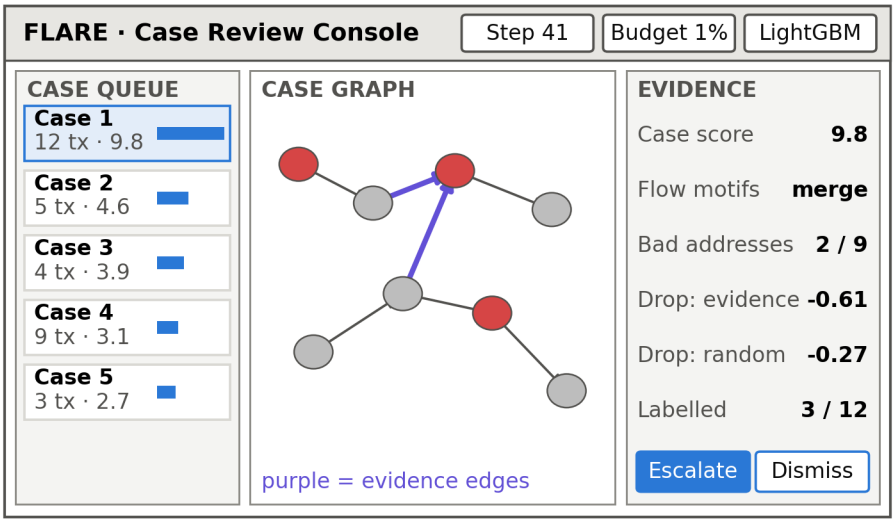


Figure 6.1. Wireframe of a case-review console: ranked cases, the case graph with counterfactual evidence edges, the evidence behind the score and the member transactions (concept for future work).

6.4 Concluding remarks

Reviewing cases instead of transactions, with the same budget and information, yields the same detections with far less fragmentation and adds detections when labels or models are weak. With the leakage audit and the open code, this gives a reproducible base for BTP-2, where validated group labels and cost models can test the economic argument directly.

References

[1] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[2] M. Weber *et al.*, “Anti-money laundering in Bitcoin: Experimenting with graph convolutional networks for financial forensics,” arXiv:1908.02591, 2019.

[3] Y. Elmougy and L. Liu, “Demystifying fraudulent transactions and illicit nodes in the Bitcoin network for financial forensics,” in *Proc. 29th ACM SIGKDD Conf. Knowledge Discovery and Data Mining (KDD)*, 2023, pp. 3979-3990, doi: 10.1145/3580305.3599803.

[4] Y. Elmougy and L. Liu, “Elliptic++ dataset: A graph network of Bitcoin blockchain transactions and wallet addresses,” GitHub repository git-disl/EllipticPlusPlus, 2023. [Online]. Available: https://github.com/git-disl/EllipticPlusPlus

[5] N. Han, R. Zhang, X. Liu, and H. Zhang, “Illicit Bitcoin transaction detection via feature-gated temporal graph learning,” *Scientific Reports*, vol. 16, Art. no. 22352, 2026, doi: 10.1038/s41598-026-53783-y.

[6] W. Zhang, Y. Chen, N. Ye, C. J. Zhang, and X. Li, “Temporal-directional tree-walk graph neural network for illicit transaction detection,” *IEEE Trans. Neural Netw. Learn. Syst.*, early access, 2026, doi: 10.1109/TNNLS.2026.3732968.

[7] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, “Leakage in data mining: Formulation, detection, and avoidance,” *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 4, pp. 1-21, 2012, doi: 10.1145/2382577.2382579.

[8] A. Malik, “Do transaction-level and actor-level AML queues agree? An empirical evaluation of granularity effects on the Elliptic++ graph,” arXiv:2604.23494, 2026.

[9] S. Choudhary, “FLARE: Flow-Linked Alert Review - code, results and documentation for this report,” GitHub repository, 2026. [Online]. Available: https://github.com/Onethybeing/flare

[10] J. Lorenz, M. I. Silva, D. Aparicio, J. T. Ascensao, and P. Bizarro, “Machine learning methods to detect money laundering in the Bitcoin blockchain in the presence of label scarcity,” in *Proc. 1st ACM Int. Conf. AI in Finance (ICAIF)*, 2020, pp. 1-8, doi: 10.1145/3383455.3422549.

[11] C. Bellei *et al.*, “The shape of money laundering: Subgraph representation learning on the blockchain with the Elliptic2 dataset,” arXiv:2404.19109, 2024.

[12] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, “A comprehensive survey on graph neural networks,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4-24, 2021, doi: 10.1109/TNNLS.2020.2978386.

[13] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2017, arXiv:1609.02907.

[14] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, arXiv:1706.02216.

[15] L. Akoglu, H. Tong, and D. Koutra, “Graph based anomaly detection and description: A survey,” *Data Mining and Knowledge Discovery*, vol. 29, no. 3, pp. 626-688, 2015, doi: 10.1007/s10618-014-0365-y.

[16] X. Ma *et al.*, “A comprehensive survey on graph anomaly detection with deep learning,” *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12012-12038, 2023, doi: 10.1109/TKDE.2021.3118815.

[17] Y. Dou, Z. Liu, L. Sun, Y. Deng, H. Peng, and P. S. Yu, "Enhancing graph neural network-based fraud detectors against camouflaged fraudsters," in *Proc. 29th ACM Int. Conf. Information and Knowledge Management (CIKM)*, 2020, pp. 315-324, doi: 10.1145/3340531.3411903.

[18] A. Pareja *et al.*, "EvolveGCN: Evolving graph convolutional networks for dynamic graphs," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 34, no. 4, 2020, pp. 5363-5370, doi: 10.1609/aaai.v34i04.5984.

[19] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, and M. Bronstein, "Temporal graph networks for deep learning on dynamic graphs," arXiv:2006.10637, 2020.

[20] Y. Shi and M. Tang, "Uncertainty-aware spatio-temporal contrastive graph neural networks for cyber financial fraud detection and risk management," *Scientific Reports*, vol. 16, Art. no. 25068, 2026, doi: 10.1038/s41598-026-55651-1.

[21] A. P. Ichull and J. C. Sopuru, "TWIN-GNN: An interpretable multi-graph framework for illicit Bitcoin transaction and wallet detection," *J. Cybersecurity and Privacy*, vol. 6, no. 6, Art. no. 186, 2026, doi: 10.3390/jcp6060186.

[22] J. Wang, J. Liu, W. Zheng, and Y. Ge, "Temporal heterogeneous graph contrastive learning for fraud detection in credit card transactions," *IEEE Access*, vol. 13, pp. 145754-145771, 2025, doi: 10.1109/ACCESS.2025.3599787.

[23] M. O. Faruq *et al.*, "Temporal and relational graph neural networks for fraud detection in transaction networks," *IEEE Open J. Comput. Soc.*, vol. 7, pp. 1439-1449, 2026, doi: 10.1109/OJCS.2026.3734644.

[24] Y. Chen, H. Zhu, H. Xu, C. Yi, Z. Chen, and D. Landa-Silva, "SF-DynFinGC: Scale-free dynamic graph construction for event-driven spatiotemporal learning in financial anomaly detection," *IEEE Trans. Artif. Intell.*, vol. 7, no. 10, pp. 5693-5706, 2026, doi: 10.1109/TAI.2026.3698049.

[25] Z. Zhang, L. Song, E. Bao, X. Lv *et al.*, "ATM-GAD: Adaptive temporal motif graph anomaly detection for financial transaction networks," arXiv:2508.20829, 2025 (preprint).

[26] Y. Jiao, H. Fan, X. Yue, W. Ping, T. Sun, and J. Wang, "Dynamic heterogeneous graph contrastive learning for uncovering collusive financial fraud," *Scientific Reports*, vol. 16, Art. no. 28993, 2026, doi: 10.1038/s41598-026-58938-5.

[27] R. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, "GNNExplainer: Generating explanations for graph neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, arXiv:1903.03894.

[28] H. Yuan, H. Yu, J. Wang, K. Li, and S. Ji, "On explainability of graph neural networks via subgraph explorations," in *Proc. 38th Int. Conf. Machine Learning (ICML)*, 2021, arXiv:2102.05152.

[29] L. Nguyen, M. Boersma, and E. Acar, "Detecting fraud in financial networks: A semi-supervised GNN approach with Granger-causal explanations," arXiv:2507.01980, 2025.

[30] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLOS ONE*, vol. 10, no. 3, Art. no. e0118432, 2015, doi: 10.1371/journal.pone.0118432.

[31] J. Davis and M. Goadrich, "The relationship between precision-recall and ROC curves," in *Proc. 23rd Int. Conf. Machine Learning (ICML)*, 2006, pp. 233-240, doi: 10.1145/1143844.1143874.

[32] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman and Hall/CRC, 1994, doi: 10.1201/9780429246593.

[33] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing*

Systems (NeurIPS), vol. 30, 2017.

[34] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5-32, 2001, doi: 10.1023/A:1010933404324.

[35] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.

[36] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, arXiv:1912.01703.

[37] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *J. Statistical Mechanics: Theory and Experiment*, vol. 2008, no. 10, Art. no. P10008, 2008, doi: 10.1088/1742-5468/2008/10/P10008.

[38] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in *Proc. 7th Python in Science Conf. (SciPy)*, 2008, pp. 11-15.

Appendix A: Hyperparameters and Tuned Configurations

Table A.1 lists the model hyperparameters, Table A.2 the as-of address features, and Table A.3 the group configurations selected on the validation period.

Table A.1. Hyperparameters.

Component	Setting
LightGBM	learning rate 0.03; 63 leaves; min. child samples 20; feature fraction 0.7; bagging 0.8 (freq. 1); L2 1.0; up to 3,000 trees with early stopping (100 rounds) on validation PR-AUC; seeds 0-4
Random forest	300 trees; 50 features per split; seeds 0-2
Logistic regression	C = 0.5; balanced class weights; signed-log transform and training-period standardisation
GraphSAGE	2 layers x 128 units; dropout 0.3; Adam (lr 0.003, weight decay 0.0005); up to 500 epochs; early stopping (patience 60 epochs) on validation PR-AUC; positive weight sqrt(n-/n+); seeds 0-4
Wallet LightGBM	same as LightGBM; three seeds; transaction risk for training rows from five-fold out-of-fold predictions
Groups	hub degree 30; size cap 40 (Louvain split, seed 0); address-link cap 10 transactions per address-step
Bootstrap	2,000 paired resamples over the 14 test steps
Random groups (E4)	20 BFS-grown connected groups per evaluated group, same size and step

Table A.2. As-of address features (each aggregated by mean and max over input and output addresses).

Feature	Meaning (address a at step t)
cur_send, cur_recv	transactions in which a is an input / output at step t
cur_ntx, cur_btc, cur_fee	distinct transactions, total BTC and fees of those transactions
prior_cur_send, prior_cur_recv, prior_cur_ntx	the same counts summed over steps $s < t$
prior_steps, age	earlier active steps; steps since first appearance
prior_ill, prior_lic	illicit / licit training labels of a's transactions in steps $s < t$, $s \leq 30$
prior_ill_frac, prior_any_ill, prior_mean_btc	illicit fraction, any illicit history, mean BTC per earlier transaction

Table A.3. Group configurations selected on validation steps 31-35 (main run).

Risk | method	Seed multiplier	q	h	Score	(β, γ, δ, base)
LightGBM|flow (ours)	1.5	0.1	2	ours	(0.5, 0.5, 0.5, sum)
LightGBM|conncomp	1.5	0.1	-	sum	-
LightGBM|ego1hop	1.5	0.05	-	mean	-
GraphSAGE|flow (ours)	1.5	0.05	2	ours	(1.0, 1.0, 0, sum)
GraphSAGE|conncomp	1.5	0.05	-	sum	-
GraphSAGE|ego1hop	1.5	0.05	-	mean	-

Appendix B: Additional Results

Table B.1 gives illicit transactions found by every queue at every budget, and Table B.2 the remaining transaction-level results. Complete tables, including all seeds, are in the results folder of the repository [9].

Table B.1. Illicit transactions found by each queue and budget (test steps 36-49).

Risk	Budget	Tx queue	FLARE	Conn. comp.	1-hop	Wallet	Tx cases/100	FLARE cases/100
LightGBM	0.5%	115	108	107	95	52	81.9	37.1
LightGBM	1.0%	208	210	205	167	96	83.7	36.4
LightGBM	2.0%	326	338	334	268	171	84.7	37.0
LightGBM	5.0%	527	547	548	421	372	79.6	54.8
GraphSAGE	0.5%	74	103	95	87	52	94.9	46.0
GraphSAGE	1.0%	146	174	181	158	95	90.9	45.8
GraphSAGE	2.0%	266	310	312	252	171	88.2	57.0
GraphSAGE	5.0%	472	494	497	370	372	79.9	63.7

Table B.2. Remaining transaction-level model results.

Model	Features	PR-AUC	ROC-AUC	Illicit F1	P@500
Random forest	AF+AUG	0.723	0.880	0.724	1.000
Logistic regr.	AF+AUG	0.276	0.874	0.341	0.344
Logistic regr.	AF	0.256	0.866	0.341	0.324

Appendix C: Software and Reproducibility

All code, results and documentation are public at https://github.com/Onethybeing/flare [9]. The repository contains a Python package (flare) with the as-of feature builder, the step graph, the group builders and scores, the review queues, bootstrap tests, the direction-aware GraphSAGE and the counterfactual test, covered by 35 unit tests; the exact Kaggle notebook sources that produced every table; all result tables as CSV and markdown; scripts that redraw every figure and diagram from the tables; and documentation of the methodology, the data audit and the reproduction steps (Table C.1). The raw Elliptic++ files are not redistributed.

Table C.1. Repository structure.

Path	Content
src/flare/	tested package: asof, graph, groups, queues, metrics, models, counterfactual
tests/	35 unit tests (graph, grouping, scores, queues, as-of features, leakage audit, bootstrap, GraphSAGE)
kaggle/main/btp_main.py	main experiment notebook source (E1-E6)
kaggle/extra/extra_body.py	supplementary experiments (X1, X2)
results/	all result tables (CSV, markdown) and run summaries
figures/, docs/assets/	result charts and diagrams (regenerated by scripts/)
docs/	METHODOLOGY, DATA_AUDIT, RESULTS, REPRODUCIBILITY
examples/quickstart.py	end-to-end example on a synthetic graph

A minimal use of the package on any transaction graph is shown below.

```
from flare import StepGraph, flow_groups, group_features, group_score, run_queue,
queue_stats, pooled
G = StepGraph(n, flow_src, flow_dst, addr_a, addr_b)
groups = flow_groups(G, nodes, risk, seed_frac=0.015, q_exp=0.1, hops=2, hub_degree=30,
max_size=40)
F = np.array([group_features(G, g, risk) for g in groups])
reviewed = run_queue(groups, group_score(F, 'sum'), risk, nodes, budget=40)
print(pooled(queue_stats(G, reviewed, labels, nodes)))
```

`python -m unittest discover -s tests` runs the tests and `python examples/quickstart.py` reproduces the synthetic example.